

# Отчёт по лабораторной работе 4

## Развёртывание приложения на удалённом сервере

Милютин Никита, БР1.2  
29 сентября 2026 г.

### 1. Цель и задание

Файлы работы находятся в БР1.2/Miliutin\_Nikita/labs/lab4. Далее относительные пути отсчитываются от каталога БР1.2/Miliutin\_Nikita.

Подготовить удалённый сервер, развернуть на нём приложение и настроить nginx в режиме обратного прокси для доступа извне. В качестве приложения используется система бронирования столиков из ЛРЗ: API Gateway, пять микросервисов и RabbitMQ.

Дополнительное условие: сохранить работу уже установленного семейного VPN. Новые компоненты должны быть отделены от WireGuard и Marzban по конфигурациям, контейнерам, сетям и портам.

### 2. Предварительная настройка

Подключение выполнялось по SSH к серверу 193.233.16.27. До изменений проверены ОС, память, свободное место, слушающие порты, Docker-сети, контейнеры и конфигурация nginx. Сохранены копия /etc/nginx, контрольная сумма существующего сайта и параметры запущенных VPN-контейнеров.

| Параметр             | Проверенное значение          |
|----------------------|-------------------------------|
| Операционная система | Ubuntu 22.04.5 LTS            |
| Оперативная память   | 1963 МиБ, без swap            |
| Docker Engine        | 29.7.2                        |
| Docker Compose       | v5.4.0                        |
| nginx                | 1.18.0, пакет Ubuntu          |
| Среда приложения     | Node.js 22 Alpine, TypeScript |
| Выделенный каталог   | /opt/br-lab4                  |

Необходимые зависимости уже были установлены и работали: повторная установка и обновление не понадобились. Node.js используется внутри контейнеров и не устанавливается на хост. Порт 80 занят существующим nginx, поэтому для лабораторной выбран свободный порт 8080. Правила firewall вручную не менялись; при создании сетей Docker добавил свои правила для нового проекта.

### 3. Архитектура развёртывания

Client / Postman

```
-> http://193.233.16.27:8080
-> nginx (host)
-> 127.0.0.1:13003 -> gateway:3003
    -> auth:4001
    -> restaurants:4002
    -> tables:4003
    -> reservations:4004 -> rabbitmq:5672
    -> notifications:4005 <- reservation.*
```

Compose-проект называется br-lab4. Сервисы общаются по именам Docker DNS. Сеть backend, подсеть 172.30.40.0/24, объявлена как internal: true. Порты микросервисов и RabbitMQ, включая Management UI, на хост не опубликованы. Только Gateway дополнительно подключён к сети proxy (172.30.41.0/24): она позволяет опубликовать порт исключительно на loopback хоста.

Путь запроса бронирования: nginx передаёт запрос Gateway; Gateway направляет его Reservation Service. Этот сервис проверяет ресторан, столик, вместимость и занятость, сохраняет бронь в памяти, публикует reservation.created. Notification Service получает событие через очередь notification-service.reservation-events, записывает уведомление в память и отправляет подтверждение ack. При отмене используется reservation.cancelled. Получить уведомления можно через Gateway.

### 4. Сборка и контейнеры

Исходники взяты из labs/lab3/src без изменения бизнес-логики. На VPS переданы src, package.json, package-lock.json, tsconfig.json и .dockerignore. Локальные node\_modules, dist, отчёты и секреты не копировались в контекст приложения.

Один образ используется для всех шести Node.js-процессов; Compose задаёт каждому собственную команду запуска. В первой стадии устанавливаются зависимости и компилируется TypeScript, во второй остаются JavaScript и зависимости выполнения.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build && npm prune --omit=dev

FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
COPY --from=build --chown=node:node /app/package.json ./package.json
USER node
CMD ["node", "dist/gateway/index.js"]
```

## 5. Запуск на VPS

Конфигурации находятся в `/opt/br-lab4/deploy`, исходники в `/opt/br-lab4/app`. Файл `.env` создан на сервере со случайным паролем RabbitMQ и правами 600. Пароль SSH и действующий пароль брокера не включены в отчёт или файлы репозитория.

```
docker build --memory=512m -t br-lab4-app:1.0.0 \
  -f /opt/br-lab4/deploy/Dockerfile /opt/br-lab4/app
cd /opt/br-lab4/deploy
docker compose config --quiet
docker compose up -d --wait --wait-timeout 180
docker compose ps
```

Запуск учитывает `depends_on` с условием `service_healthy`: потребитель уведомлений должен создать очередь до запуска сервиса бронирований. Gateway запускается после готовности остальных API. HTTP-проверки контролируют доступность процессов; они не заменяют сквозной тест с публикацией события.

Для Node.js-контейнеров установлены лимиты 128 МиБ и 0,40 CPU, для RabbitMQ 384 МиБ и 0,50 CPU. Ограничены число процессов и размер логов. Node.js работает не от `root`; корневая файловая система контейнеров приложения доступна только для чтения. Временные файлы разрешены в отдельном `tmpfs`.

Политика `restart: unless-stopped` позволяет Docker перезапускать упавшие контейнеры и запускать ранее работавшие контейнеры после старта демона. Перезагрузка VPS в ходе работы не выполнялась, чтобы не прерывать VPN. Полное автоматическое восстановление AMQP-соединений после сбоя не реализовано.

RabbitMQ использует отдельный именованный том `br-lab4_rabbitmq-data`. Он сохраняет состояние брокера, но не массивы пользователей, бронирований и уведомлений внутри Node.js.

## 6. Настройка nginx

Файл лабораторной добавлен отдельно от существующего сайта:

```
/etc/nginx/sites-available/br-lab4.conf
/etc/nginx/sites-enabled/br-lab4.conf -> sites-available/br-lab4.conf
```

После подключения выполнены `nginx -t` и `systemctl reload nginx`. Проверка синтаксиса завершилась успешно. Применено мягкое перечитывание конфигурации, без перезапуска Docker или VPN-контейнеров.

`proxy_pass` направляет запрос на локальный Gateway. Поскольку в адресе `proxy_pass` нет дополнительного URI, исходный путь, например `/api/restaurants`, передаётся дальше. Заголовки сохраняют исходный Host, IP клиента, протокол и идентификатор запроса. X-Lab: BR-LR4 позволяет проверить, что ответ прошёл через новый блок nginx.

## 7. Конфигурация обратного прокси

```
# Loaded by /etc/nginx/sites-enabled/br-lab4.conf in the http context.
limit_req_zone $binary_remote_addr zone=br_lab4_requests:10m rate=15r/s;

server {
    listen 8080;
    server_name 193.233.16.27;
    server_tokens off;
    client_max_body_size 64k;

    access_log /var/log/nginx/br-lab4.access.log;
    error_log /var/log/nginx/br-lab4.error.log warn;

    add_header X-Lab "BR-LR4" always;
    add_header X-Content-Type-Options nosniff always;

    location / {
        limit_req zone=br_lab4_requests burst=60 nodelay;
        limit_req_status 429;
        proxy_pass http://127.0.0.1:13003;
        proxy_http_version 1.1;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Request-ID $request_id;
        proxy_set_header Connection "";
        proxy_connect_timeout 5s;
        proxy_send_timeout 15s;
        proxy_read_timeout 15s;
        proxy_hide_header X-Powered-By;
    }
}
```

Лимит 15 запросов в секунду на IP с кратковременным всплеском до 60 запросов уменьшает риск случайной перегрузки учебного API. Тело запроса ограничено 64 КиБ; заданы таймауты подключения и ответа upstream. Доступ и ошибки записываются в отдельные файлы `br-lab4.access.log` и `br-lab4.error.log`.

Публичный адрес: `http://193.233.16.27:8080`. Ответ корневого маршрута:

```
{"message": "Containerized Restaurant Booking API Gateway is running"}
```

Сервис предоставляет JSON API. Графический сайт и Swagger в рамках этой лабораторной не разворачиваются; коллекция Postman ЛР3 используется с новым базовым адресом.

## 8. Проверка работоспособности

С внешней машины запущен `verify.mjs`, использующий встроенные `fetch` и `assert` Node.js. Скрипт проверяет коды HTTP, содержимое JSON и заголовок `nginx`, создаёт отдельного тестового пользователя и собственную бронь, а затем отменяет её. Результаты сохранены в `verification.json`.

```
node labs/lab4/verify.mjs http://193.233.16.27:8080 \
  labs/lab4/verification.json
```

Время завершения успешного прогона (UTC): 2026-09-29T11:00:47.620Z. Итог: **28 проверок пройдено, 0 ошибок**.

| Группа проверок   | Результат                                                      |
|-------------------|----------------------------------------------------------------|
| Gateway и nginx   | Ответ 200, заголовок X-Lab; неизвестный маршрут 404            |
| Пользователи      | Регистрация 201, вход 200, профиль; ошибки 400/401/409         |
| Каталог           | Рестораны, фильтры, детали и столики; отсутствующий объект 404 |
| Бронирование      | Создание 201; проверки вместимости и обязательных полей        |
| Права и конфликты | Без токена 401, чужая отмена 403, занятый слот 409             |
| Отмена и история  | Отмена 200, статус cancelled, отсутствующая бронь 404          |
| RabbitMQ          | Получены reservation.created и reservation.cancelled           |

События сопоставлялись с конкретными пользователем и номером новой брони. Для каждого уведомления проверялось наличие `source_event_id`. Это проверяет реальное прохождение события через издателя, брокер и потребителя, а не только доступность HTTP-маршрутов.

После обработки событий проверена очередь:

|                                         |       |                |           |
|-----------------------------------------|-------|----------------|-----------|
| name                                    | ready | unacknowledged | consumers |
| notification-service.reservation-events | 0     | 0              | 1         |

Все семь контейнеров имели статус `healthy`. Отдельная внешняя TCP-проверка портов 13003, 3003, 4001–4005, 5672 и 15672 вернула `ECONNREFUSED`; напрямую эти компоненты извне недоступны. При замере контейнеры приложения и брокер суммарно занимали около 233 МиБ, на VPS было доступно около 1 ГиБ памяти. Это наблюдение после проверки, не результат нагрузочного тестирования.

## 9. Сохранность существующих служб

Сравнение до и после показало неизменность ID, времени запуска, количества перезапусков и состояния контейнеров `wg-easy`, `marzban-node` и `marzban-marzban-1`. Старый сайт `nginx:80` и панель `WireGuard:51821` ответили HTTP 200. Контрольная сумма `sites-available/default` совпала. Записи проверки сохранены в `server-status.txt`, исходное сравнение на VPS в `/opt/br-lab4/evidence`. Прохождение трафика через VPN с клиентского устройства отдельно не тестировалось.

## 10. Обнаруженное ограничение и восстановление

Первый сквозной прогон выявил ошибку 500 при создании брони: после пересоздания RabbitMQ старый AMQP-канал в Reservation Service закрылся. Процессы при этом продолжали отвечать на HTTP-healthcheck. В исходном приложении нет автоматического reconnect.

После готовности RabbitMQ отдельно пересозданы только Notifications и Reservations. Затем весь сценарий повторён успешно. Для явных обновлений брокера через Compose добавлен `depends_on.restart: true`; этот параметр не обеспечивает восстановление канала при произвольном сбое сети или автоматическом перезапуске брокера самим Docker.

Команды ручного восстановления после готовности RabbitMQ:

```
cd /opt/br-lab4/deploy
docker compose up -d --no-deps --force-recreate --wait notifications
docker compose up -d --no-deps --force-recreate --wait reservations
```

Пересоздание теряет данные соответствующих сервисов в памяти. Для production необходимы постоянное хранилище, восстановление AMQP-соединений, подтверждение публикаций и согласование записи брони с отправкой события, например outbox.

## 11. Ограничения учебного развёртывания

- PostgreSQL отсутствует. Пользователи, бронирования и уведомления находятся в массивах Node.js и теряются при перезапуске соответствующего процесса.
- Токен user-N не является подписанным JWT; пароль хранится без хеширования. Список уведомлений доступен без авторизации. Настоящие данные использовать нельзя.
- Доступ организован по HTTP. HTTPS, сертификат и доменное имя в этом развёртывании не настраивались.
- Существующий lockfile при сборке дал предупреждение npm audit о двух уязвимостях: одной low и одной moderate. Зависимости исходного задания не менялись; перед промышленным использованием требуется отдельное обновление.
- Проверены функциональные сценарии, но не отказоустойчивость, нагрузка или полная безопасность приложения.

## 12. Вывод

Приложение LP3 развёрнуто на удалённом VPS в отдельном Compose-проекте. nginx обеспечивает внешний доступ через порт 8080, внутренние компоненты не опубликованы в Интернет. Выполнены 28 успешных сквозных проверок, включая обмен событиями через RabbitMQ. Существующие VPN-контейнеры не перезапускались и не перенастраивались. Конфигурации, инструкция и результаты проверки собраны в `labs/lab4`; копия исходников находится на сервере.

## 13. Источники

- nginx: ngx\_http\_proxy\_module.
- Docker Compose: порядок запуска и зависимости.
- Docker Compose: сетевое взаимодействие.
- Исходники проекта `labs/lab3/src` и конфигурации `labs/lab4`.